CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 12

Event Management CRUD with Poster Upload and File Validation

PHP 8 · MySQL (MariaDB) · MySQLi · Bootstrap 5

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

1. Problem Definition

StudentHub needs an event module in which administrators publish events with a poster image. Uploading files is one of the most dangerous features of a web application: if the server trusts the file name, the extension or the browser's `accept` attribute, an attacker can upload a PHP script and run it, an SVG file with JavaScript, a disguised executable or a huge file. A carelessly built edit or delete function can also destroy the wrong file, follow a path such as `../db.php`, or lose the old poster when an update fails. The module must therefore validate every poster on the server, store it safely, change files only after the database change succeeded, and let only authorised administrators manage events.

Starting point: the Practical-12 folder contained a simple teaching version that used the shared `event_db` database, a `users` table with a role column, checked only the extension and `getimagesize()`, had no CSRF protection and stored posters in a web-accessible folder. It was backed up and replaced by the application described here, which uses its own database `studenthub_prac12` and its own administrator table.

2. Objectives

- Add, view, update and delete events (title, description, category, date, time, venue, poster, status).
- Upload event posters and validate the **file type** (JPEG, PNG, WebP) and **file size** (max 2 MB) on the server.
- Show clear success and failure messages for every action.
- Restrict event management to authorised administrators with a secure login, session handling and logout.
- Intermediate extension: event **categories** and **Open/Closed** status filtering.
- Use MySQLi prepared statements, output escaping and CSRF protection throughout.

3. Tools and Technologies

Tool	Use in this practical
PHP 8.2.12 (XAMPP)	Application pages, sessions, password_verify(), upload handling
MySQLi	All database access with prepared statements
PHP Fileinfo extension	Detects the real MIME type of an uploaded file from its content
MariaDB 10.4.32 (XAMPP)	Database studenthub_prac12; export with mysqldump
Bootstrap 5.3.3	Responsive layout, forms, tables, alerts (stored locally, no CDN)
HTML5, CSS3, JavaScript	Forms with HTML5 validation; poster preview before upload (app.js)
Microsoft Edge	Running the application; headless Edge (Playwright) for the browser test and screenshots
Python 3	tests/test_app.py – automated end-to-end tests (standard library only)

4. Database Design

The practical uses its own database, `studenthub_prac12`, with three InnoDB tables. No other practical's database is used.

Table / column	Type	Notes
events.event_id	INT UNSIGNED AUTO_INCREMENT	Primary key
events.event_title	VARCHAR(150)	CHECK at least 3 characters
events.description	TEXT	Required (10–2000 characters, checked by the application)
events.category	VARCHAR(40)	FOREIGN KEY →

		event_categories.category_name; index
events.event_date / event_time	DATE / TIME	Composite index idx_events_date
events.venue	VARCHAR(150)	Required
events.poster_filename	VARCHAR(64)	UNIQUE; CHECK REGEXP ^[a-f0-9]{32}[.](jpg|png|webp)$
events.status	ENUM('open','closed')	Default 'open'; index
events.created_at / updated_at	DATETIME	updated_at changes on every UPDATE
event_categories	category_name PK, description	Cultural, Hackathon, Seminar, Sports, Technical, Workshop
admins	admin_id, full_name, email UNIQUE, password_hash, created_at, last_login_at	CHECK: password_hash must be a bcrypt/Argon2 hash (no plaintext)

The foreign key keeps the category list in the database (the drop-downs are filled from `event_categories`). The UNIQUE key on `poster_filename` guarantees that two events can never share one file, so deleting one event can never remove another event's poster, and the CHECK constraint allows only the generated safe names – a value such as `../db.php` cannot be stored at all.

`studenthub_prac12.sql` is the mysqldump export: 6 categories, **12 fictional events (8 open, 4 closed)** whose poster files are in `uploads/`, and the fictional administrator `admin.demo@example.com`. Its password is stored only as a `password_hash()` hash of a random password that is not in the repository; the password is set locally with `tools\set_admin_password.php`.

studenthub_prac12.sql (lines 106–126) – events table from the dump

```
106  CREATE TABLE `events` (
107    `event_id` int(10) unsigned NOT NULL AUTO_INCREMENT,
108    `event_title` varchar(150) NOT NULL,
109    `description` text NOT NULL,
110    `category` varchar(40) NOT NULL,
111    `event_date` date NOT NULL,
112    `event_time` time NOT NULL,
113    `venue` varchar(150) NOT NULL,
114    `poster_filename` varchar(64) NOT NULL,
115    `status` enum('open','closed') NOT NULL DEFAULT 'open',
116    `created_at` datetime NOT NULL DEFAULT current_timestamp(),
117    `updated_at` datetime NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp(),
118    PRIMARY KEY (`event_id`),
119    UNIQUE KEY `uq_events_poster_filename` (`poster_filename`),
120    KEY `idx_events_category` (`category`),
121    KEY `idx_events_status` (`status`),
122    KEY `idx_events_date` (`event_date`,`event_time`),
123    CONSTRAINT `fk_events_category` FOREIGN KEY (`category`) REFERENCES `event_categories`
      (`category_name`) ON UPDATE CASCADE,
124    CONSTRAINT `chk_events_poster_filename` CHECK (`poster_filename` regexp '^[a-f0-
      9]{32}[.](jpg|png|webp)$'),
125    CONSTRAINT `chk_events_title` CHECK (char_length(`event_title`) >= 3)
126  ) ENGINE=InnoDB AUTO_INCREMENT=13 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1 Setting up and running the application

- Start MySQL in the XAMPP Control Panel.
- Import the database: phpMyAdmin → Import → `studenthub_prac12.sql`, or `C:\xampp\mysql\bin\mysql.exe -u root < studenthub_prac12.sql` in the Practical-12 folder.
- Set the demo password: `C:\xampp\php\php.exe tools\set_admin_password.php admin.demo@example.com` (type 8–72 characters).
- Start the server: double-click `start-server.bat`, or run `C:\xampp\php\php.exe -S 127.0.0.1:8612 router.php` in the Practical-12 folder.

- Open **http://127.0.0.1:8612/login.php** and log in with `admin.demo@example.com`.
- Optional settings (database, upload folder, timeout): copy `config.example.php` to the git-ignored `config.local.php`.

5. Application Implementation

File	Purpose
login.php / logout.php	Administrator login (password_verify, session ID regeneration) and POST-only logout
index.php	READ – event table with poster thumbnails, category and Open/Closed filters, messages
view.php	Event details with the full-size poster
add.php / edit.php	CREATE / UPDATE – validation, poster upload or replacement
delete.php	DELETE – confirmation page (GET) and deletion of the event and its poster (POST)
poster.php	The only way to view a poster: validated name, image Content-Type, admins only
includes/functions.php	Session, security headers, CSRF token, require_admin(), messages, validation, layout
includes/upload.php	Poster validation, random file names, safe file paths and deletion
tools/*.php	Command line: set the admin password; list/delete abandoned poster files
router.php, .htaccess, uploads/.htaccess	Block private files and the upload folder (HTTP 403)

5.1 Administrator authentication

The login form posts email, password and a CSRF token. The email is looked up with a prepared statement and the password is checked with `password_verify()`; an unknown email is checked against a dummy hash so that both failures take about the same time and show the same message, *"Invalid email or password."* After a successful login `session_regenerate_id(true)` gives the session a new ID (protection against session fixation) and a new CSRF token. The session cookie `P12SESSID` is HttpOnly and SameSite=Strict, and a session ends after 30 minutes of inactivity.

login.php (lines 28–46) – password verification and session regeneration

```
28          $stmt = get_db()->prepare('SELECT admin_id, full_name, email, password_hash FROM admins WHERE
    email = ?');
29          $stmt->bind_param('s', $email);
30          $stmt->execute();
31          $row = $stmt->get_result()->fetch_assoc();
32          // An unknown email is checked against a dummy hash, so both failures take about the same time
33          $hash = $row['password_hash'] ??
    '$2y$10$qAzAl/cRo1Zi1CMFWjkd6uXqfnlXMCb8ADDDWVXGEfMWXNVeKN7Pa';
34          $verified = password_verify($password, $hash);          // always runs, also for an unknown
    email
35          if ($verified && $row !== null && $email !== '' && $password !== '') {
36              session_regenerate_id(true);
37              $_SESSION = [
38                  'admin'         => ['id' => (int) $row['admin_id'], 'name' => $row['full_name'],
    'email' => $row['email']],
39                  'last_activity' => time(),
40                  'csrf_token'    => bin2hex(random_bytes(32)),       // new token for the new session
41              ];
42              $update = get_db()->prepare('UPDATE admins SET last_login_at = NOW() WHERE admin_id = ?');
43              $update->bind_param('i', $_SESSION['admin']['id']);
44              $update->execute();
45              flash('success', 'Welcome, ' . $row['full_name'] . '. You are logged in as
    administrator.');
46              redirect('index.php');
```

Every event page – including `poster.php` – starts with `require_admin()`. Without a valid administrator session the server answers **HTTP 401 "Access denied"** before any data is read or changed.

includes/functions.php (lines 100–122) – access control for all event pages

```
100  function require_admin(): array
101  {
102      start_app();
103      $admin = current_admin();
104      if ($admin !== null && time() - (int) ($_SESSION['last_activity'] ?? 0) >
      session_timeout_seconds()) {
105          end_session();
106          redirect('login.php?status=timeout');
107      }
108      if ($admin === null) {
109          http_response_code(401);
110          page_start('Access denied');
111          echo '<div class="card border-danger shadow-sm mx-auto access-card">', "\n",
112               '  <div class="card-body p-4">', "\n",
113               '    <h2 class="h4 text-danger mb-3">Access denied</h2>', "\n",
114               '    <p class="mb-2">Event management is restricted to authorized administrators. You are
      not logged in, so this page was refused by the server (HTTP 401).</p>', "\n",
115               '    <p class="text-secondary small mb-4">No event data was shown or changed.</p>', "\n",
116               '    <a class="btn btn-primary" href="login.php">Go to administrator login</a>', "\n",
117               '  </div>', "\n", '</div>', "\n";
118          page_end();
119          exit;
120      }
121      $_SESSION['last_activity'] = time();
122      return $admin;
```

6. CRUD Functionality

6.1 Create and read

`add.php` validates all fields (`validate_event()`: required values, lengths, a category from the table, a real date, a time HH:MM, status open/closed) and the poster (Section 7). Only when everything is valid is the poster stored under a random name and the event inserted with a prepared INSERT; if the INSERT fails, the stored file is removed again. `index.php` lists all events with a poster thumbnail, title, category, date and time, venue and an Open (green) or Closed (grey) badge; the new event is highlighted after adding.

The category and status filters accept only values from the category table and the two statuses; the WHERE clause is built from fixed pieces with `?` placeholders.

index.php (lines 13–33) – category and Open/Closed filter

```
 13  // WHERE clause from fixed pieces with ? placeholders; the values were checked against the allowed
      lists above
 14  $where = [];
 15  $types = '';
 16  $params = [];
 17  if ($category !== '') {
 18      $where[] = 'category = ?';
 19      $types .= 's';
 20      $params[] = $category;
 21  }
 22  if ($status !== '') {
 23      $where[] = 'status = ?';
 24      $types .= 's';
 25      $params[] = $status;
 26  }
 27  $stmt = $db->prepare('SELECT event_id, event_title, category, event_date, event_time, venue,
      poster_filename, status FROM events'
 28      . ($where ? ' WHERE ' . implode(' AND ', $where) : '') . ' ORDER BY event_date, event_time,
      event_id');
 29  if ($types !== '') {
 30      $stmt->bind_param($types, ...$params);
 31  }
 32  $stmt->execute();
 33  $events = $stmt->get_result()->fetch_all(MYSQLI_ASSOC);
```

6.2 Update with poster replacement

Without a new file the existing poster is kept. A replacement is validated first and stored under a **new** random name; then the UPDATE runs, and only after it succeeded is the old file deleted. If the UPDATE fails, the new file is removed and the old poster stays in MySQL and on disk. A rejected replacement changes nothing at all.

edit.php (lines 43–63) – safe poster replacement

```
43              $new_name = $replace ? store_poster($poster) : null;
44              if ($replace && $new_name === null) {
45                  $errors['poster'] = 'The new poster could not be saved on the server. The current
    poster was kept.';
46              } else {
47                  $poster_name = $new_name ?? $stored['poster_filename'];
48                  try {
49                      $stmt = $db->prepare('UPDATE events SET event_title = ?, description = ?, category
    = ?, event_date = ?, event_time = ?,
50                                            venue = ?, poster_filename = ?, status = ? WHERE event_id =
    ?');
51                      $stmt->bind_param('ssssssssi', $event['event_title'], $event['description'],
    $event['category'], $event['event_date'],
52                                        $event['event_time'], $event['venue'], $poster_name,
    $event['status'], $id);
53                      $stmt->execute();
54                  } catch (Throwable $exception) {
55                      discard_new_poster($new_name);
56                      throw $exception;
57                  }
58                  if ($new_name !== null) {
59                      delete_poster_file($db, $stored['poster_filename']);   // the old file is no
    longer used by any event
60                  }
61                  flash('success', 'Event “' . $event['event_title'] . '” was updated successfully'
62                      . ($new_name !== null ? ' and its poster was replaced.' : '; the existing poster
    was kept.'), $id);
63                  redirect('index.php');
```

6.3 Delete with confirmation

Delete opens `delete.php?id=N`, which shows the poster and details and asks *“Are you sure you want to delete …?”*. Only the POST from **Yes, delete event** (with the CSRF token) deletes the row; afterwards its poster file is removed through `delete_poster_file()`.

delete.php (lines 22–29) – delete the row, then its poster

```
22      $stmt = $db->prepare('DELETE FROM events WHERE event_id = ?');
23      $stmt->bind_param('i', $id);
24      $stmt->execute();
25      if ($stmt->affected_rows !== 1) {
26          flash('error', 'The event could not be deleted. Please try again.');
27          redirect('index.php');
28      }
29      $file_removed = delete_poster_file($db, $event['poster_filename']);
```

7. Poster Upload and Validation

All checks run on the server in `validate_poster()`; the `accept` attribute of the file field is only a convenience. Accepted are **JPEG, PNG and WebP images up to 2 MB** (and up to 6000 × 6000 px):

Step	Check	Rejects for example
1	PHP upload error code	partial upload, file above the server limit, missing temp folder
2	is_uploaded_file()	a path that was not uploaded in this request

3	size 1 byte – 2 MB	empty files, files over 2 MB
4	extension .jpg/.jpeg/.png/.webp	.gif, .svg, .php, .exe
5	Fileinfo MIME type of the content, matching the extension	PHP script named .jpg, SVG named .png, PNG named .jpg
6	getimagesize: same image type, 1–6000 px	fake images, gigantic dimensions
7	complete structure (PNG chunks + CRC to IEND, JPEG end marker, WebP RIFF length)	truncated or damaged images
8	no <?php inside the file	a valid JPEG with appended PHP code

includes/upload.php (lines 59–92) – content-based checks

```
 59      $extension = strtolower(pathinfo((string) ($file['name'] ?? ''), PATHINFO_EXTENSION));
 60      if (!isset(POSTER_EXTENSIONS[$extension])) {
 61          return ['ok' => false, 'error' => 'Invalid file type. Only JPEG, PNG and WebP images (.jpg,
     .jpeg, .png, .webp) are allowed.'];
 62      }
 63
 64      // The file is read once and every content check works on these bytes. Reading can fail when
     security software
 65      // on the server blocks a malicious upload; that file is simply rejected.
 66      $content = @file_get_contents($tmp);
 67      if ($content === false || strlen($content) !== $size) {
 68          error_log('Practical 12: an uploaded file could not be read (blocked or removed)');
 69          return ['ok' => false, 'error' => 'Invalid file type. The file could not be read as an
     image.'];
 70      }
 71
 72      $mime = (new finfo(FILEINFO_MIME_TYPE))->buffer($content);
 73      if (!in_array($mime, POSTER_EXTENSIONS, true)) {
 74          return ['ok' => false, 'error' => 'Invalid file type. The file content is not a JPEG, PNG or
     WebP image.'];
 75      }
 76      if ($mime !== POSTER_EXTENSIONS[$extension]) {
 77          return ['ok' => false, 'error' => 'The file extension does not match its content (.' .
     $extension . ' file containing ' . $mime . ').'];
 78      }
 79
 80      $info = @getimagesizefromstring($content);
 81      if ($info === false || !isset(POSTER_IMAGE_TYPES[$info[2]]) || image_type_to_mime_type($info[2])
     !== $mime
 82          || !image_structure_ok($content, POSTER_IMAGE_TYPES[$info[2]])) {
 83          return ['ok' => false, 'error' => 'The poster is not a valid image or is damaged.'];
 84      }
 85      if ($info[0] < 1 || $info[1] < 1 || $info[0] > POSTER_MAX_PIXELS || $info[1] > POSTER_MAX_PIXELS)
     {
 86          return ['ok' => false, 'error' => 'The poster must be between 1 and ' . POSTER_MAX_PIXELS . '
     pixels wide and high.'];
 87      }
 88
 89      if (stripos($content, '<?php') !== false) {
 90          return ['ok' => false, 'error' => 'The poster was rejected because it contains embedded
     code.'];
 91      }
 92      return ['ok' => true, 'tmp' => $tmp, 'ext' => POSTER_IMAGE_TYPES[$info[2]]];
```

Safe storage. The original name is never used: `store_poster()` names the file with 32 random hexadecimal characters (`random_bytes(16)`) plus the extension of the *detected* type and moves it with `move_uploaded_file()`. Only this name is stored in MySQL. The `uploads/` folder is not reachable over HTTP – the router and `uploads/.htaccess` answer 403, switch the PHP engine off and remove script handlers – so even a file that contained code could never run. Posters are delivered by `poster.php`, which accepts only names matching `^[a-f0-9]{32}\.(jpg|png|webp)$` inside the upload folder and sends them as images with `nosniff` and a sandbox CSP.

includes/upload.php (lines 151–177) – no path traversal, no deleting other events' posters

```
151  function poster_path(string $name): ?string
152  {
153      if (!preg_match(POSTER_NAME_PATTERN, $name)) {
```

```
154         return null;
155     }
156     $dir = realpath(upload_dir());
157     $path = $dir === false ? false : realpath($dir . DIRECTORY_SEPARATOR . $name);
158     if ($path === false || dirname($path) !== $dir || !is_file($path)) {
159         return null;
160     }
161     return $path;
162 }
163
164 // Deletes a poster file, but only a valid name inside the upload folder that no event uses any more
165 // (poster_filename is also UNIQUE in MySQL, so two events can never share one file).
166 function delete_poster_file(mysqli $db, ?string $name): bool
167 {
168     if ($name === null || ($path = poster_path($name)) === null) {
169         return false;
170     }
171     $stmt = $db->prepare('SELECT COUNT(*) FROM events WHERE poster_filename = ?');
172     $stmt->bind_param('s', $name);
173     $stmt->execute();
174     if ((int) $stmt->get_result()->fetch_row()[0] > 0) {
175         return false;
176     }
177     return @unlink($path);
```

No abandoned files. Rejected uploads are never moved out of PHP's temporary folder (PHP deletes them after the request); a stored file is removed again when the INSERT or UPDATE fails; `tools\cleanup_orphan_posters.php` lists and (with `--delete`) removes valid-named files that no event uses and that are older than one hour.

8. Security Measures

- **SQL injection:** every query uses `prepare()` with placeholders and `bind_param()`; the tests confirm there is no `query()` call.
- **XSS:** every value is printed through `e()` = `htmlspecialchars(…, ENT_QUOTES, 'UTF-8')`; Content-Security-Policy `default-src 'self'`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`.
- **CSRF:** a random 64-character token per session in every form, checked with `hash_equals()`; add, edit, delete and logout accept only POST.
- **Sessions:** strict mode, HttpOnly + SameSite=Strict cookie, ID regenerated at login, 30-minute timeout, logout destroys the session and the cookie, `Cache-Control: no-store` on protected pages.
- **Uploads:** Section 7 – server-side validation, random names, non-public and non-executable folder, safe replacement and deletion.
- **Errors and configuration:** MySQL errors are logged and replaced by *"The event database is not available right now."*; credentials belong in the git-ignored `config.local.php`; `db.php`, `includes/`, `tools/`, `tests/`, `uploads/`, the SQL dump and documents return HTTP 403.

9. Output Screenshots

All screenshots are genuine captures of the running application in Microsoft Edge (headless, 1280 px wide) taken during the browser test, with the demo database studenthub_prac12 and fictional data.

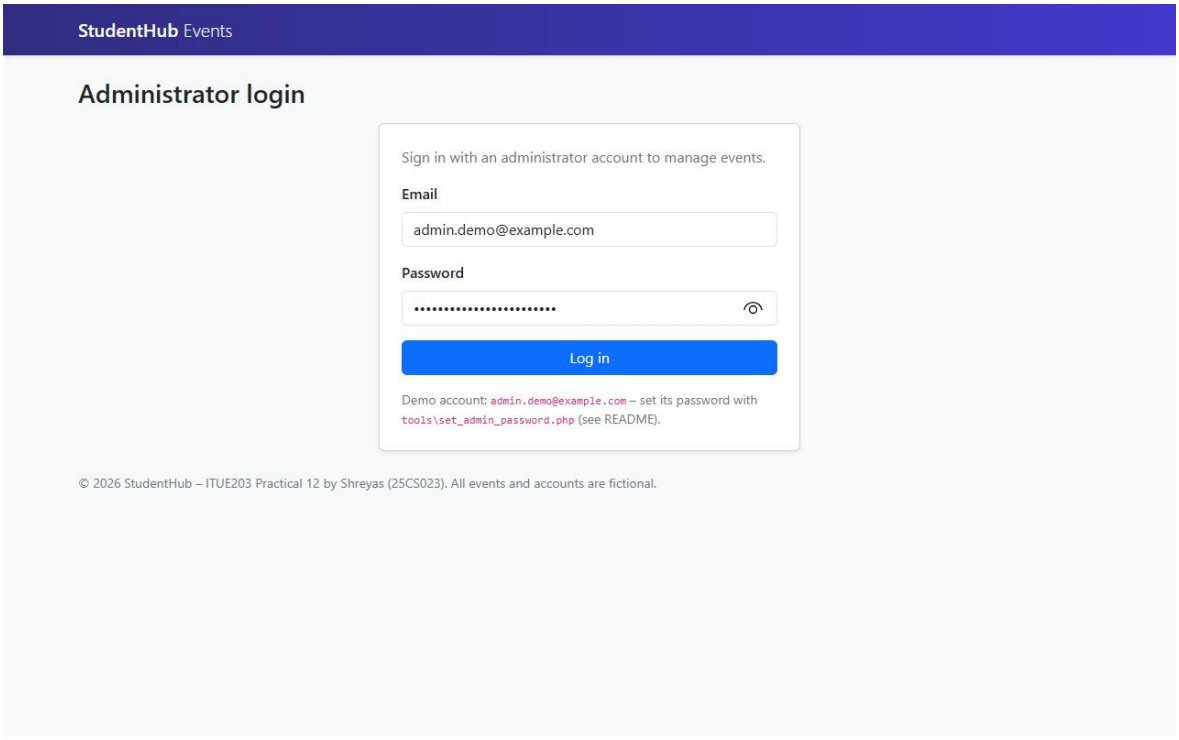


Figure 1: Administrator login page (email and password entered) (admin-login.png)

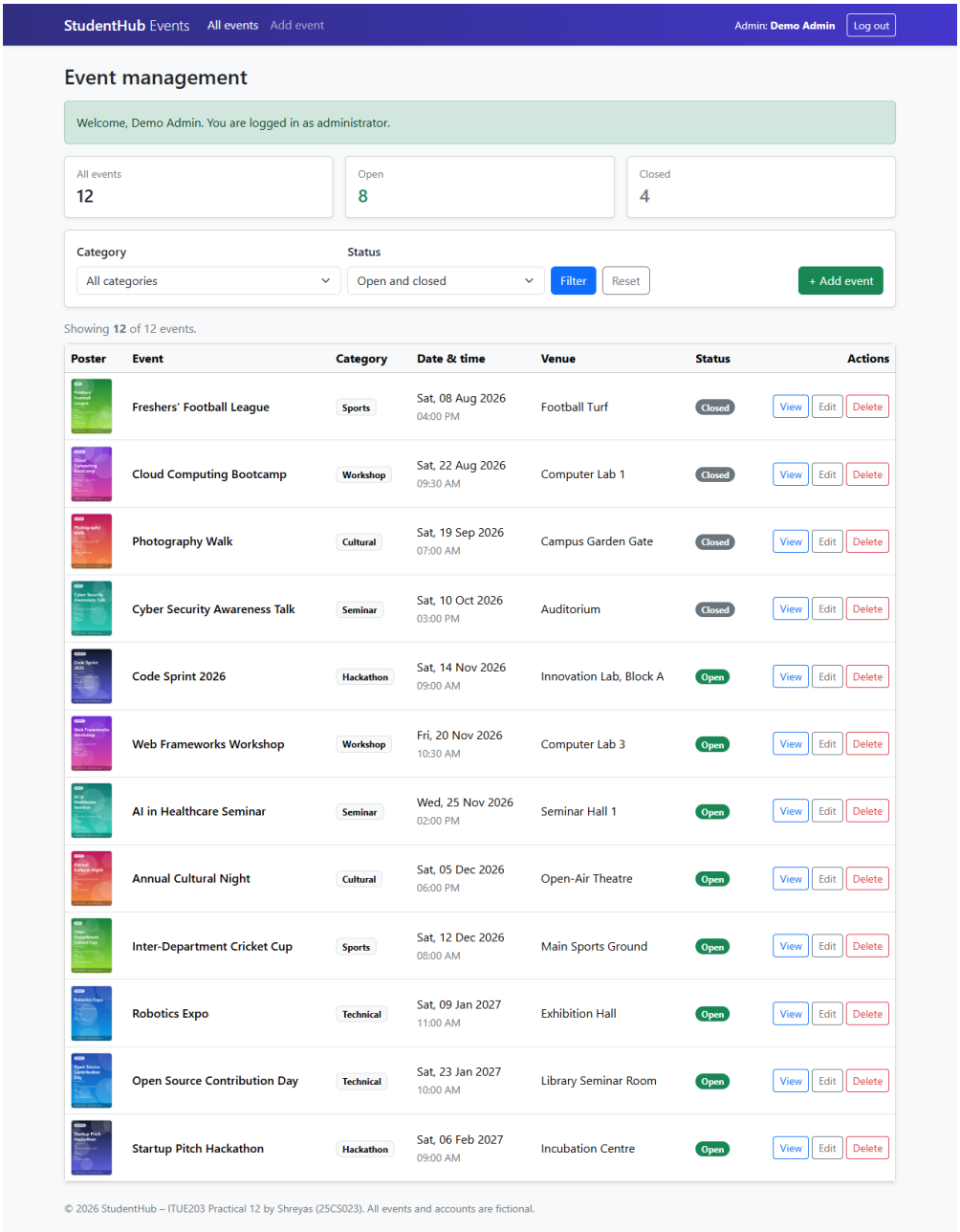


Figure 2: Event list after login – 12 events with poster thumbnails, category, date, venue and Open/Closed status (events-list.png)

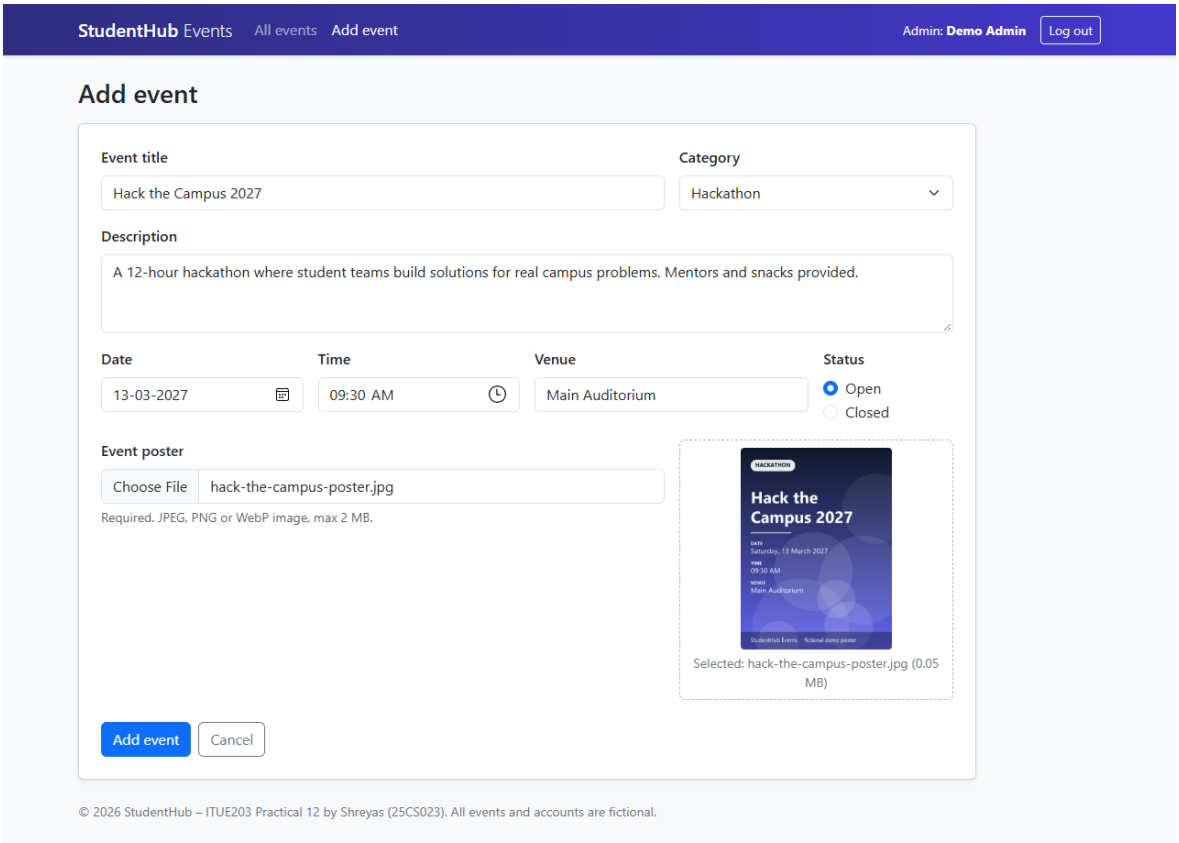


Figure 3: Add event form – all fields filled and a JPEG poster selected (preview shown before upload) (add-event-form.png)

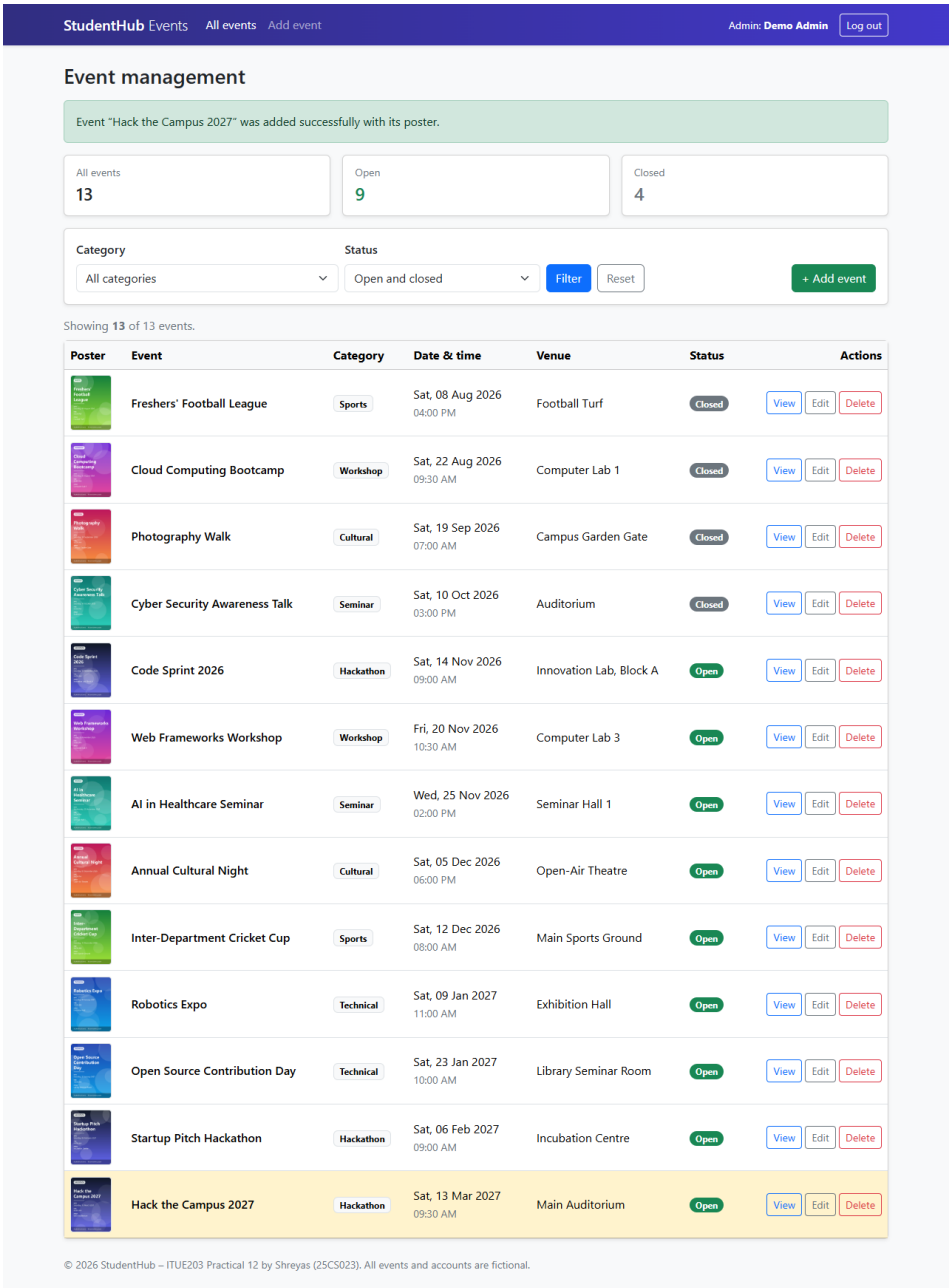


Figure 4: Successful addition – message and the new event "Hack the Campus 2027" highlighted with its poster (successful-event-add.png)

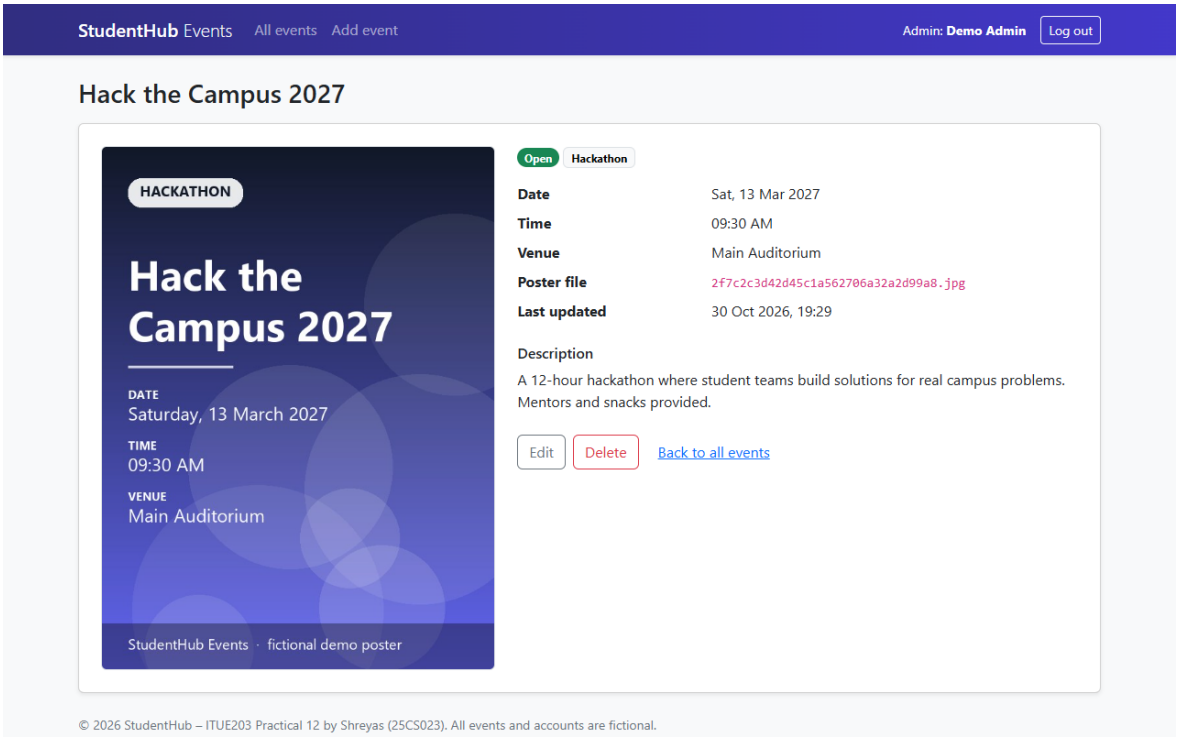


Figure 5: Uploaded poster displayed on the event page, stored under a random 32-character name (event-poster-preview.png)

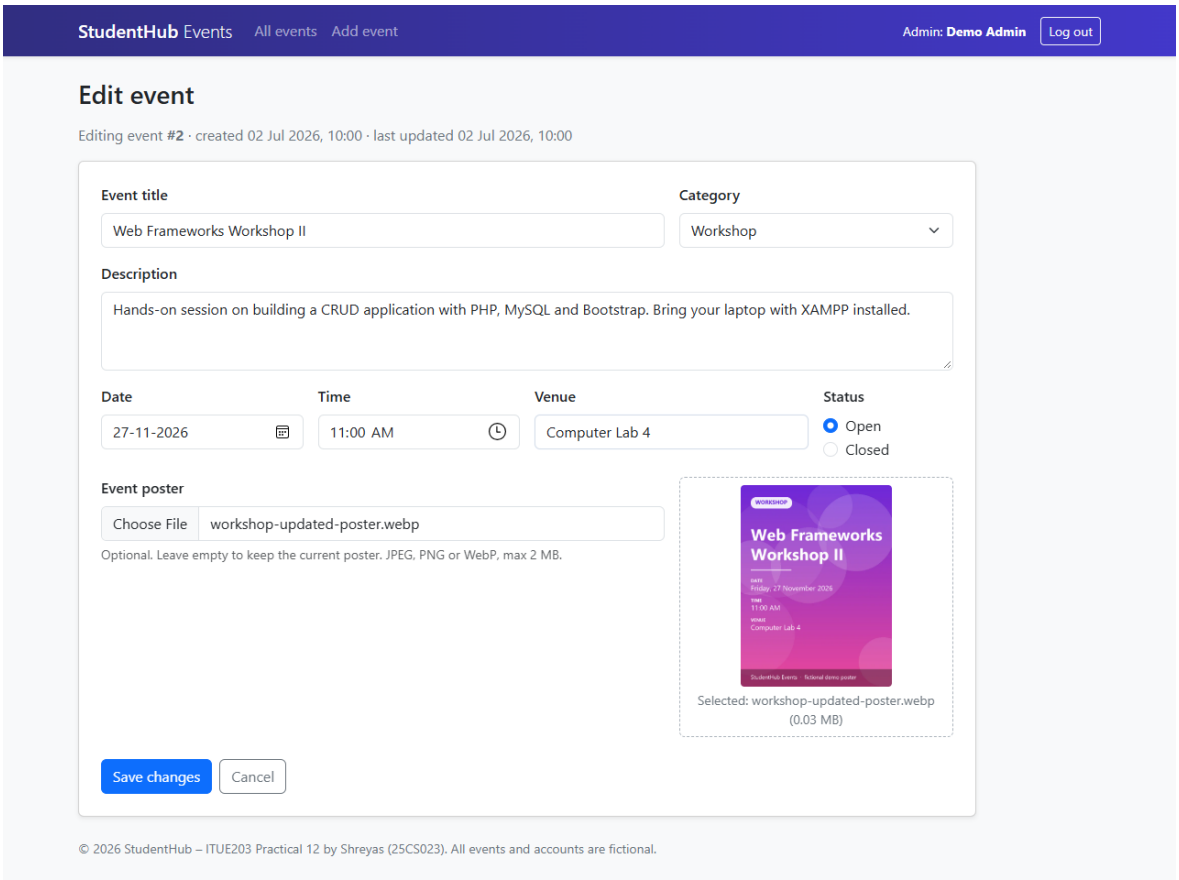


Figure 6: Editing event #2 – new title, date, time and venue and a replacement WebP poster (edit-event-form.png)

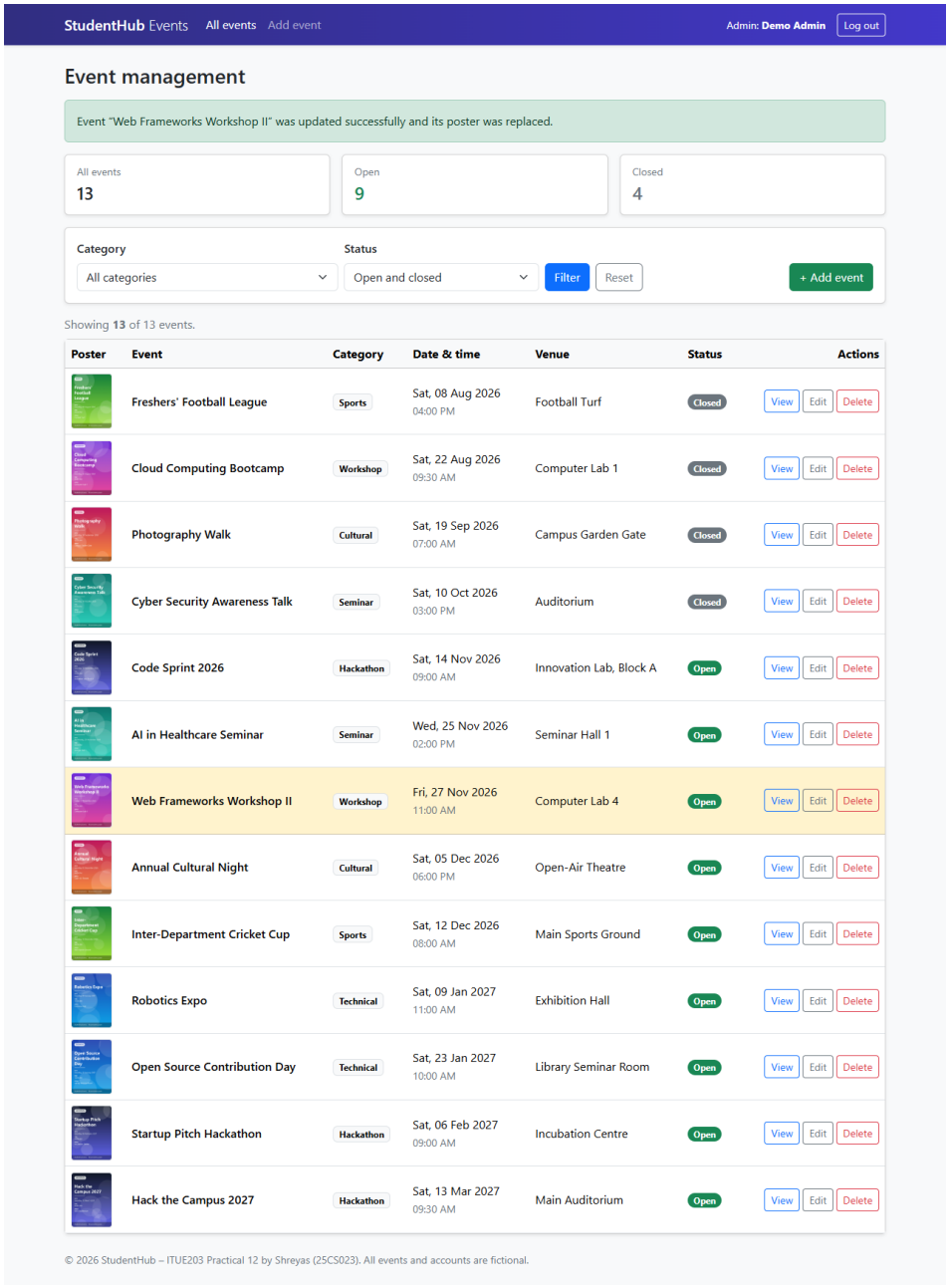


Figure 7: Successful update – "…updated successfully and its poster was replaced" (successful-event-update.png)

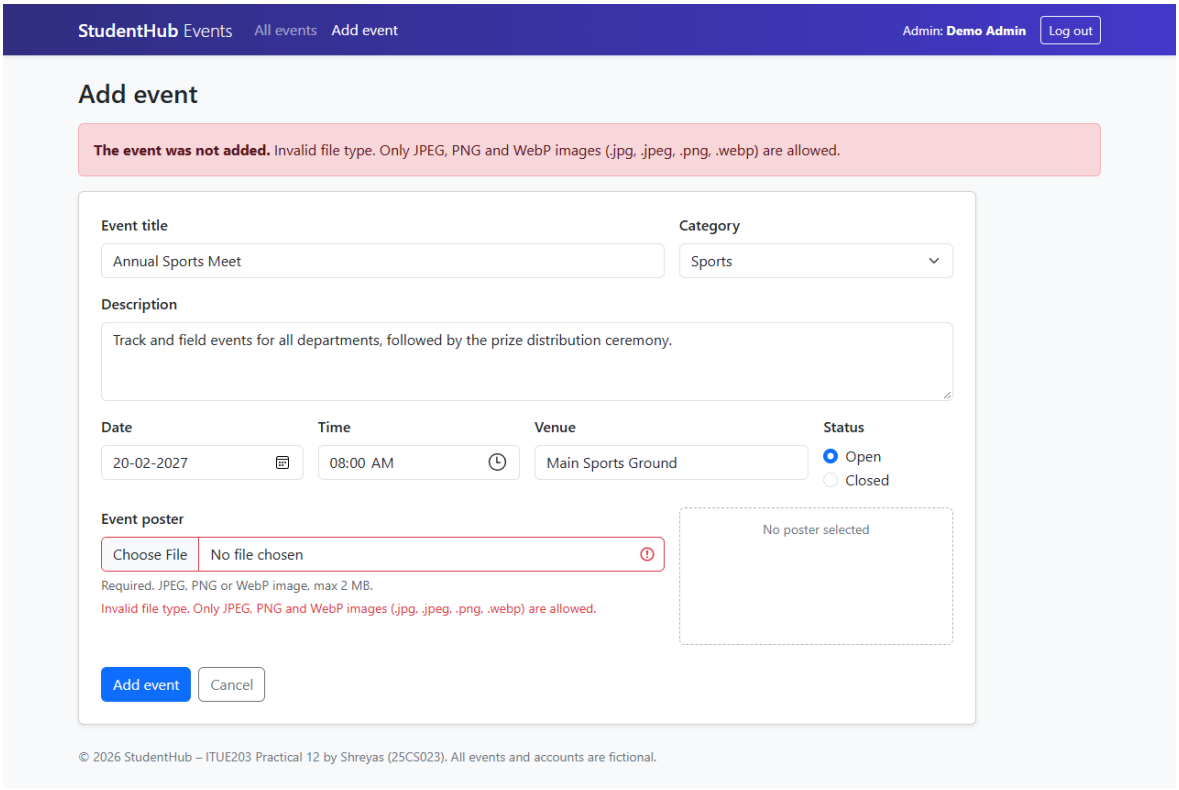


Figure 8: Invalid file type – an SVG file is rejected by the server (invalid-file-type.png)

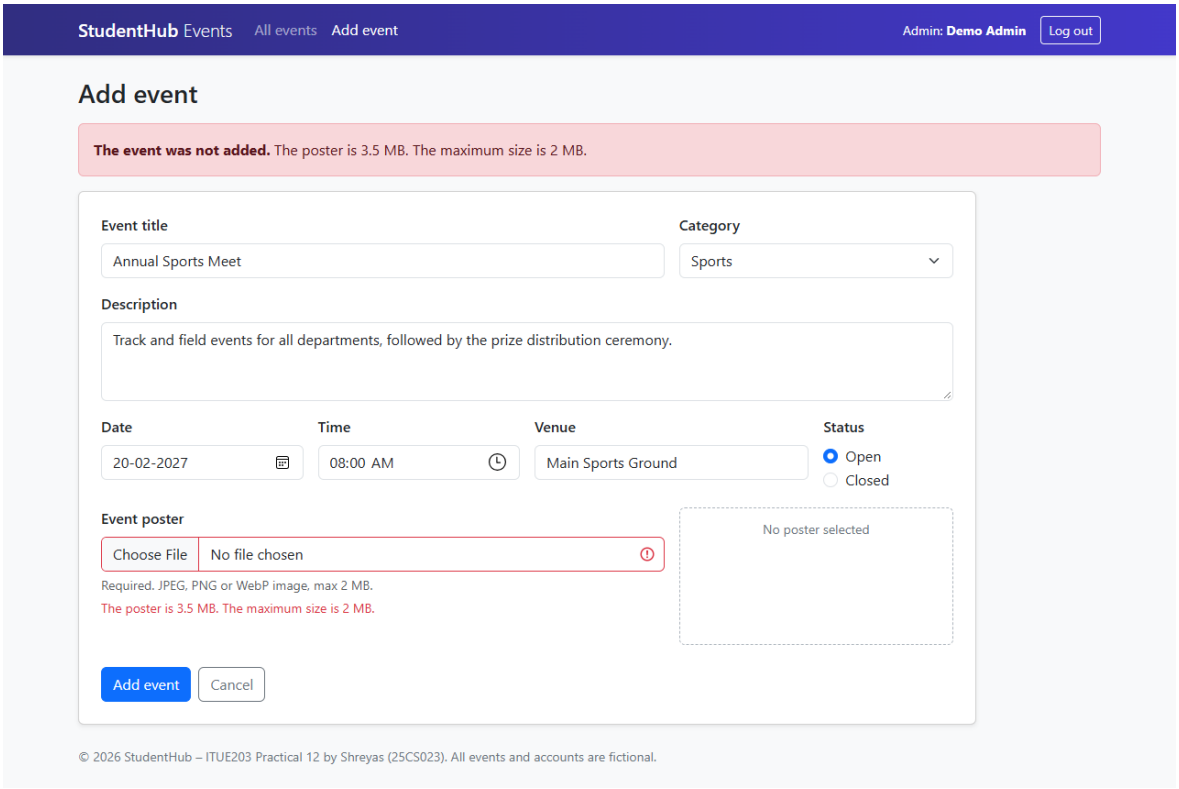


Figure 9: File size rejection – a 3.5 MB poster is rejected (maximum 2 MB) (file-size-rejection.png)

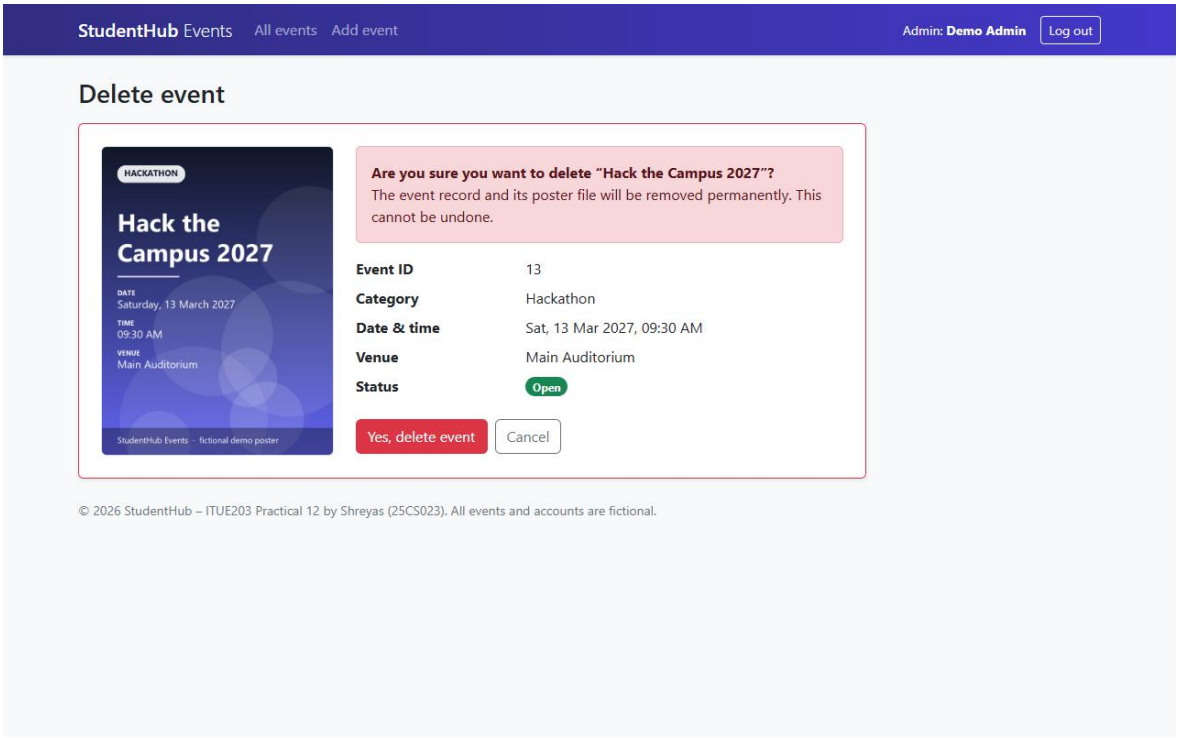


Figure 10: Confirmation before deleting "Hack the Campus 2027" (delete-confirmation.png)

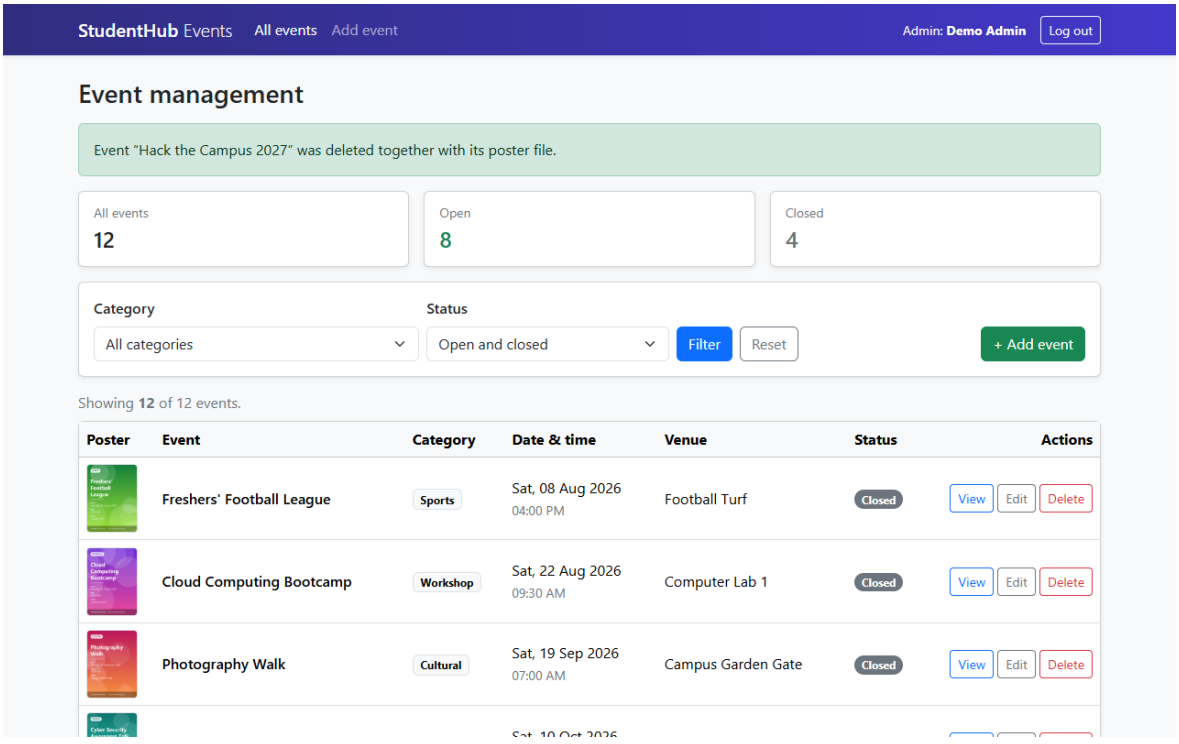


Figure 11: Successful deletion – "…was deleted together with its poster file" (successful-event-delete.png)

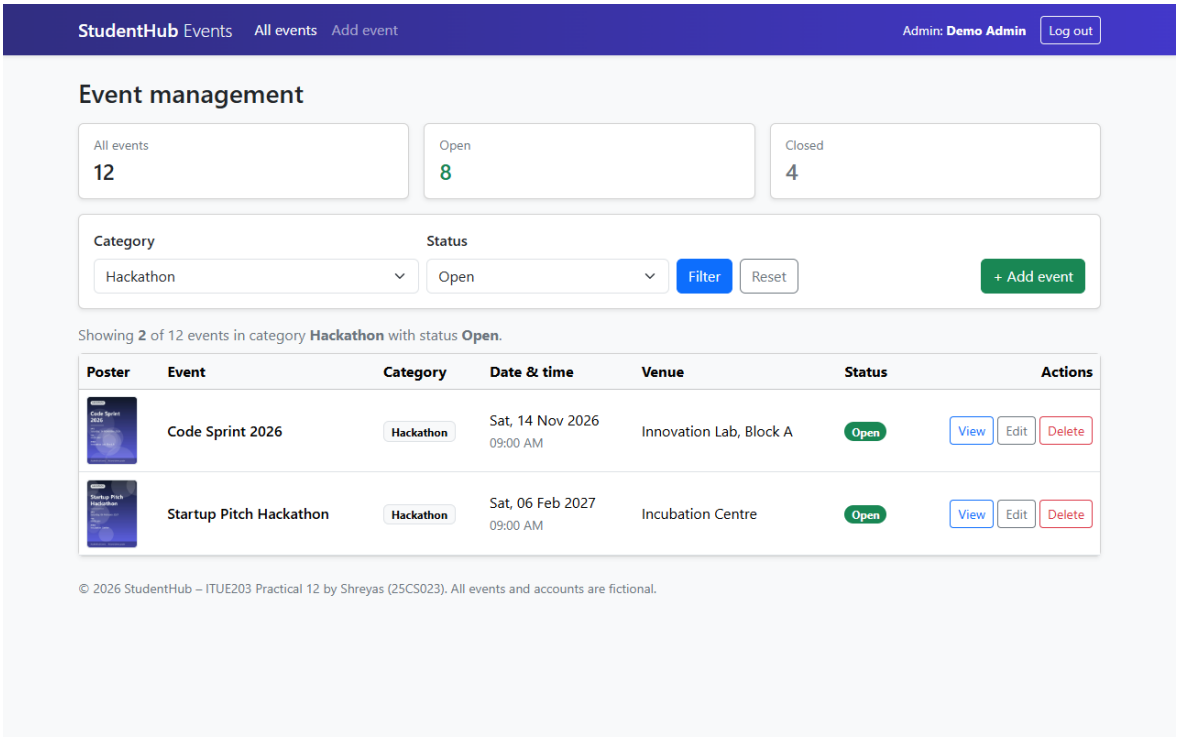


Figure 12: Category and status filters – Hackathon + Open shows 2 of 12 events (category-status-filter.png)

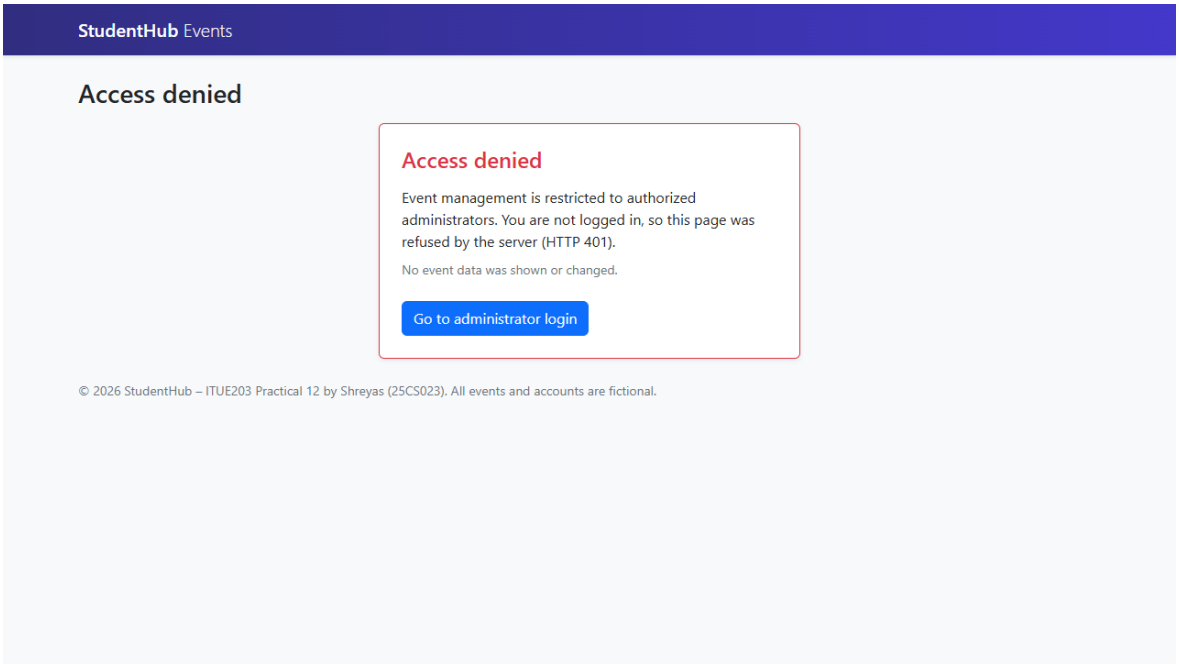


Figure 13: Not logged in – index.php refused with HTTP 401 "Access denied" (unauthorized-access-denied.png)

10. Test Cases and Results

`tests/test_app.py` (Python standard library) creates the disposable database `studenthub_prac12_test` and a temporary upload folder, sets a random admin password, starts three PHP servers (normal; 2-second timeout with small upload limits; MySQL unreachable), sends real HTTP and multipart upload requests and checks the pages, the database and the files. Test triggers in the test database force an INSERT and an UPDATE to fail after a poster was stored, to prove that no file is left behind and no old poster is lost. Everything is removed at the end; `studenthub_prac12` and `uploads/` are not changed.

Result: 153 of 153 automated checks passed, 0 failed. `php -l` found no syntax errors in the 15 PHP files. The browser test in Microsoft Edge passed **13 of 13** checks.

ID	Test case	Expected result	Result
TC01	Database connection and seed data	12 events (8 open, 4 closed), 6 categories, 1 hashed admin; page loads	Pass
TC02	Constraints	refused by FK, CHECK, UNIQUE and CHECK constraints	Pass
TC03	Admin login	303 → event list, “Welcome, Demo Admin”, session ID changed, old ID useless	Pass
TC04	Failed login	HTTP 401 “Invalid email or password.” / “login form expired”; no session	Pass
TC05	Unauthorized access	HTTP 401 “Access denied”, no event data, nothing changed	Pass
TC06	Logout	only the valid POST logs out; afterwards 401, old cookie useless	Pass
TC07	Session timeout	redirect to login “Your session expired…”, session gone	Pass
TC08	Event listing	12 rows with poster thumbnails, title, category, date & time, venue, Open/Closed badge	Pass
TC09	Poster delivery	exact image bytes, image Content-Type, `nosniff`, sandbox CSP	Pass
TC10	Create with valid JPEG / PNG / WebP	303, success message, new row, file stored under `<32 hex>.<ext>` with identical bytes	Pass
TC11	Random file names	three different random names; nothing written outside `uploads/`	Pass
TC12	Required-field validation	HTTP 422, error for title, description, category, date, time, venue and poster	Pass
TC13	Tampered values	all refused	Pass
TC14	Invalid extension	“Invalid file type. Only JPEG, PNG and WebP…”	Pass
TC15	Disguised files	rejected (content is not a JPEG, PNG or WebP image)	Pass
TC16	MIME mismatch	“The file extension does not match its content”	Pass
TC17	Corrupted images	“The poster is not a valid image or is damaged.”	Pass
TC18	Embedded code	“rejected because it contains embedded code”	Pass
TC19	Oversized file	“The poster is … MB. The maximum size is 2 MB.”; exactly 2 MB is accepted	Pass
TC20	Server limits	clear “too large” messages, not a CSRF error	Pass
TC21	Image dimensions; empty file	rejected	Pass
TC22	No abandoned files	no new row and no new file in `uploads/`	Pass
TC23	Edit without new poster	“existing poster was kept”, same file name and file	Pass
TC24	Poster replacement	new random name stored, new file present, old file deleted after the update	Pass
TC25	Rejected replacement	HTTP 422, “current poster was kept”, nothing changed, no new file	Pass
TC26	Failed update	503 general message; new file removed; old poster still in MySQL and on disk	Pass
TC27	Delete confirmation	“Are you sure you want to delete …?”, nothing deleted yet	Pass
TC28	Delete and poster clean-up	refused without token; confirmed: row and only its poster file removed	Pass
TC29	Delete edge cases	“not found”, nothing deleted	Pass
TC30	Category filter	only Workshop events	Pass
TC31	Open/Closed filter	only matching events; filters stay selected	Pass
TC32	Filter tampering	ignored, all events listed	Pass
TC33	SQL injection	stored literally, table intact; not found	Pass
TC34	XSS escaping	escaped in list, view, edit and delete pages	Pass
TC35	CSRF protection	refused, nothing changed	Pass
TC36	Path traversal	HTTP 404, no file content revealed	Pass
TC37	Upload-folder execution protection	HTTP 403; planted code delivered as raw bytes, never executed	Pass
TC38	Protected files	HTTP 403	Pass
TC39	Abandoned-file clean-up tool	list mode lists it; `--delete` removes only that file	Pass
TC40	Raw database errors hidden	503 “The event database is not available right now”, details only in the PHP log	Pass

11. Problems Encountered and Fixes

- **Truncated images:** the first test run accepted a PNG cut after its header, because `getimagesize()` reads only the header. GD (which could re-encode the image) is not enabled in this XAMPP, so a structure check was added (PNG chunks with CRC up to IEND, JPEG end marker, WebP RIFF length) together with tests for truncated JPEG, PNG and WebP files.
- **Unreadable malicious upload:** for the test file containing a PHP web-shell line, `finfo::file()` logged a PHP warning – the file could not even be opened (most likely blocked by the security software). The upload is now read once; an unreadable file is rejected cleanly and all checks work on the read bytes.
- **Login timing:** the dummy-hash comparison was skipped for unknown emails because of short-circuit evaluation; `password_verify()` now always runs.
- **Content-Security-Policy:** the Edge test showed that `img-src 'self'` blocked Bootstrap's built-in data: icons (select arrows, radio dots); `data:` images are now allowed. Lazy-loaded thumbnails were removed so every poster loads at once.
- **SQL CHECK constraint:** the backslash in the poster-name REGEXP was lost, so `.` matched any character; it is now `[.]`.
- **Demo data:** the screenshot run replaced a seed poster; `uploads/` was restored and the dump re-imported afterwards.

12. Conclusion

The event management module lets authorised administrators add, view, update and delete events, filter them by category and Open/Closed status, and see clear success and error messages. Posters are accepted only after eight server-side checks – upload status, size, extension, Fileinfo MIME type, image type and dimensions, file structure and embedded code – are stored under random names in a folder that cannot be opened or executed, and are replaced or deleted only after the database change succeeded. Prepared statements, output escaping, CSRF tokens and a hardened session protect the application; all 153 automated checks and 13 browser checks passed. Possible extensions are a login-attempt limit, administrator management in the interface and image re-encoding once GD is available.

13. References

- PHP Manual – Handling file uploads: https://www.php.net/manual/en/features.file-upload.php
- PHP Manual – Fileinfo (finfo): https://www.php.net/manual/en/book.fileinfo.php
- PHP Manual – getimagesize: https://www.php.net/manual/en/function.getimagesize.php
- PHP Manual – password_verify: https://www.php.net/manual/en/function.password-verify.php
- PHP Manual – MySQLi prepared statements: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
- OWASP – File Upload Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- OWASP – Cross-Site Request Forgery Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- Bootstrap 5.3 documentation: https://getbootstrap.com/docs/5.3/
- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 (folder Practical-12)